

# 义乌小商品出海智能体-OPC

## 部署指南

---

参赛赛道：2026"直通乌镇"全球互联网大赛 OPC 特色赛

创始人：冯亦根（浙江省首批产业教授、高级工程师、工信部认证首席数据官）

日期：2026 年 6 月

## 一、GitHub 推送流程

### 1. 初始化仓库

```
cd D:\YiWuInternetCompetition
git init
git add .
git commit -m "Initial commit"
```

### 2. 关联远程仓库并推送

```
git remote add origin https://github.com/<your-username>/YiWuInternetCompetition.git
git branch -M main
git push -u origin main
```

### 3. 后续推送

```
git add .
git commit -m "描述本次改动"
git push
```

推送到 **main** 分支会自动触发部署流程（需先配置好 GitHub Secrets）。

## 二、Render 后端部署步骤

### 1. 创建 Render 账号

访问 [render.com](https://render.com)，使用 GitHub 账号登录。

### 2. 创建 Web Service

#### 1. 点击 **New** → **Web Service**

#### 2. 连接 GitHub 仓库

#### 3. 配置如下：

- **Name:** `yiwu-global-api`
- **Root Directory:** `demo`
- **Runtime:** Python
- **Build Command:** `pip install -r requirements.txt`
- **Start Command:** `uvicorn app.main:app --host 0.0.0.0 --port $PORT`

### 3. 配置环境变量

在 Render 控制台的 **Environment** 页面添加以下变量：

| 变量名                    | 说明           | 示例值                                                                                                               |
|------------------------|--------------|-------------------------------------------------------------------------------------------------------------------|
| LLM_BASE_URL           | LLM API 地址   | <a href="https://dashscope.aliyuncs.com/compatible-mode/v1">https://dashscope.aliyuncs.com/compatible-mode/v1</a> |
| LLM_API_KEY            | LLM API 密钥   | sk-xxxx                                                                                                           |
| LLM_MODEL              | 模型名称         | qwen-plus                                                                                                         |
| LLM_DAILY_LIMIT        | 每日调用上限       | 500                                                                                                               |
| DASHSCOPE_WORKSPACE_ID | 百炼工作空间 ID    | ws-xxxx                                                                                                           |
| API_SECRET             | API 签名密钥     | 自定义随机字符串                                                                                                          |
| JWT_SECRET             | JWT 鉴权密钥     | 自定义随机字符串                                                                                                          |
| ALLOWED_ORIGINS        | CORS 允许的前端域名 | <a href="https://your-app.vercel.app">https://your-app.vercel.app</a>                                             |

#### 4. 获取 Deploy Hook

4. 进入 Render 服务设置页面
5. 找到 **Deploy Hook** 部分
6. 创建一个 Deploy Hook, 复制 URL
7. 将此 URL 保存到 GitHub Secrets 中的 `RENDER_DEPLOY_HOOK_URL`

#### 5. 使用 `render.yaml` (Blueprint 部署)

项目已包含 `demo/render.yaml` 配置文件, 可直接使用 Render Blueprint 一键部署:

8. 在 Render 控制台选择 **Blueprint**
9. 连接 GitHub 仓库
10. Render 会自动读取 `render.yaml` 配置
11. 手动填写标记为 `sync: false` 的环境变量

### 三、Vercel 前端部署步骤

#### 1. 创建 Vercel 账号

访问 [vercel.com](https://vercel.com), 使用 GitHub 账号登录。

## 2. 导入项目

12. 点击 **Add New** → **Project**

13. 选择 GitHub 仓库

14. 配置如下：

- **Framework Preset:** Vite
- **Root Directory:** `demo/web`
- **Build Command:** `npm run build`
- **Output Directory:** `dist`

## 3. 配置环境变量

在 Vercel 项目设置的 **Environment Variables** 页面添加：

| 变量名                            | 说明        | 示例值                                               |
|--------------------------------|-----------|---------------------------------------------------|
| <code>VITE_API_BASE_URL</code> | 后端 API 地址 | <code>https://yiwu-global-api.onrender.com</code> |

## 4. 获取部署凭证

在 Vercel 项目设置中获取以下信息，保存到 GitHub Secrets：

- `VERCEL_TOKEN`：在 Vercel 账号设置 → Tokens 中创建
- `VERCEL_ORG_ID`：在 `.vercel/project.json` 中获取
- `VERCEL_PROJECT_ID`：在 `.vercel/project.json` 中获取

## 5. `vercel.json` 配置说明

项目已包含 `demo/web/vercel.json`，配置了：

- **productionBranch:** `main` — 仅 `main` 分支触发生产部署
- **rewrites:** SPA 路由支持 — 所有路径重写到 `index.html`

# 四、Docker 部署步骤

## 1. 构建并启动服务

```
cd demo
docker-compose up -d --build
```

## 2. 查看运行状态

```
docker-compose ps
```

```
docker-compose logs -f
```

### 3. 停止服务

```
docker-compose down
```

### 4. 配置环境变量

在 `demo/` 目录下创建 `.env` 文件（参考 `.env.example`）：

```
LLM_BASE_URL=https://dashscope.aliyuncs.com/compatible-mode/v1
LLM_API_KEY=your-api-key-here
LLM_MODEL=qwen-plus
LLM_DAILY_LIMIT=500
DASHSCOPE_WORKSPACE_ID=your-workspace-id
API_SECRET=your-secret-key
JWT_SECRET=your-jwt-secret
ALLOWED_ORIGINS=http://localhost:5173
```

### 5. 访问服务

- 后端 API: `http://localhost:8000`
- 前端页面: `http://localhost:5173`

## 五、环境变量配置清单

### 后端环境变量

| 变量名                    | 必填 | 说明         | 默认值       |
|------------------------|----|------------|-----------|
| PORT                   | 否  | 服务端口       | 8000      |
| HOST                   | 否  | 监听地址       | 0.0.0.0   |
| LLM_BASE_URL           | 是  | LLM API 地址 | -         |
| LLM_API_KEY            | 是  | LLM API 密钥 | -         |
| LLM_MODEL              | 否  | 模型名称       | qwen-plus |
| LLM_DAILY_LIMIT        | 否  | 每日调用上限     | 500       |
| DASHSCOPE_WORKSPACE_ID | 否  | 百炼工作空间 ID  | -         |

|                 |   |           |   |
|-----------------|---|-----------|---|
| API_SECRET      | 否 | API 签名密钥  | - |
| JWT_SECRET      | 否 | JWT 鉴权密钥  | - |
| ALLOWED_ORIGINS | 否 | CORS 允许域名 | - |

## 前端环境变量

| 变量名               | 必填 | 说明        | 默认值 |
|-------------------|----|-----------|-----|
| VITE_API_BASE_URL | 是  | 后端 API 地址 | -   |

## GitHub Secrets

| Secret 名称              | 说明                     |
|------------------------|------------------------|
| RENDER_DEPLOY_HOOK_URL | Render Deploy Hook URL |
| VERCEL_TOKEN           | Vercel 访问令牌            |
| VERCEL_ORG_ID          | Vercel 组织 ID           |
| VERCEL_PROJECT_ID      | Vercel 项目 ID           |

## 六、CI/CD 流程说明

CI 流程（`.github/workflows/ci.yml`）

触发条件: push 或 pull\_request 到 `main` / `dev` 分支

执行步骤:

### 15. 后端检查

- 安装 Python 3.11
- 安装依赖
- flake8 代码风格检查
- pytest 单元测试

## 16. 前端检查

- 安装 Node.js 18
- 安装依赖
- TypeScript 类型检查 (`tsc --noEmit`)
- ESLint 代码检查
- 构建验证 (`npm run build`)

CD 流程 (`\.github/workflows/deploy.yml``)

触发条件: push 到 `main` 分支

执行步骤:

## 17. 测试阶段 (与 CI 相同的检查)

- Python 单元测试
- TypeScript 编译检查

## 18. 后端部署

- 通过 Render Deploy Hook 触发后端重新部署

## 19. 前端部署

- 使用 Vercel CLI 拉取配置
- 构建前端项目
- 部署到 Vercel 生产环境

两个部署任务并行执行，都依赖测试阶段通过。

## 常见问题

### Q: Render 部署失败怎么办?

检查 Render 控制台的日志，常见原因包括：环境变量未配置、依赖安装失败、端口绑定错误。

### Q: Vercel 部署后页面空白?

确认 `vercel.json` 中的 `rewrites` 配置正确，以及 Root Directory 设置为 `demo/web`。

### Q: Docker 中前端无法连接后端?

确保 `VITE_API_BASE_URL` 指向后端容器的正确地址，Docker 环境下使用 `http://localhost:8000`。

### Q: 如何回滚部署?

- **Render:** 在部署历史中选择之前的版本重新部署

- **Vercel**: 在部署列表中选择之前的版本进行回滚